MOTOR INSURANCE CLAIMS & POLICY ANALYTICS

Standardized Internship Project Specification

Purpose: A common project interface will be used by every student. Students must follow the required folder structure, file names, table names, function names, business questions, KPIs, dashboard pages, testing requirements, and deployment flow. The implementation inside the functions and the findings from the assigned data must be the student's own work.

Prepared & Designed By

Mr. Ravi Varma
Data Science Trainer (Ex-IBM Technical Trainer)
Naresh IT, Hyderabad

1. PROJECT STANDARD

Item	Standard
Domain	Motor Insurance
Primary focus	Data Analytics + Business Analytics + Dashboard + Deployment
Data design	Multiple related CSV tables
Core tools	Python, Pandas, NumPy, Matplotlib, Seaborn, Jupyter, Streamlit
Engineering	Modular Python, validation, cleaning pipeline, logging, pytest
DevOps	Git, Docker, AWS ECR, AWS IAM, AWS EC2
Machine Learning	Not required
Final output	Live Streamlit application + project repository + required documentation
Submission	Student submits the AWS live application URL in the Google Form provided by the admin

2. BUSINESS SCENARIO

You are working as a Data Analyst / Business Analyst for a motor insurance company. The company manages customers, insurance policies, vehicles, claims, and claim payments. Management wants a reliable analytical solution to understand policy activity, premium collections, claim behavior, claim severity, settlement performance, customer patterns, and operational risk indicators.

Business questions

- How many policies are active, expired, cancelled, and renewed?
- Which policy types generate the most premium?
- How many claims are registered, approved, rejected, and pending?
- What is the total and average claim amount?
- Which vehicle types generate more claims?
- Which regions have higher claim frequency or claim severity?
- Which customer segments generate higher claim values?
- What is the average claim settlement time?
- Which claim types are most common?
- How does premium compare with claim cost?
- Which policy types have higher claim ratios?

- Which age groups or vehicle groups show different claim patterns?
- Which patterns require business attention?

3. PROJECT OBJECTIVE

Convert raw motor-insurance operational data into validated, cleaned, business-ready data; perform EDA and statistical analysis; calculate insurance KPIs; identify patterns; convert evidence into business insights and recommendations; expose the results through a Streamlit dashboard; test the code; containerize it; and deploy the application to AWS EC2 using Docker and Amazon ECR.

4. PROJECT FLOW

CLIENT / BUSINESS REQUIREMENT
↓
BRD + FRD
↓
RAW RELATED TABLES
↓
DATA LOADING
↓
RELATIONSHIP + DATA QUALITY VALIDATION
↓
DATA CLEANING
↓
REUSABLE CLEANING PIPELINE
↓
CLEANED TABLES
↓
EDA + STATISTICS
↓
INSURANCE KPI ANALYSIS
↓
BUSINESS QUESTIONS
↓
PATTERN DETECTION
↓
BUSINESS INSIGHTS
↓
RECOMMENDATIONS
↓
STREAMLIT FRONTEND
↓
PYTEST + LOGGING
↓
GIT
↓
DOCKER
↓
AWS ECR
↓
AWS IAM
↓
AWS EC2
↓
LIVE STREAMLIT URL
↓
GOOGLE FORM SUBMISSION

5. STANDARD PROJECT FOLDER STRUCTURE

```
motor_insurance_analytics/
|
├── data/
|   ├── raw/
|   |   ├── customers.csv
|   |   ├── policies.csv
|   |   ├── vehicles.csv
|   |   ├── claims.csv
|   |   └── payments.csv
|   |
|   └── cleaned/
|       ├── customers_cleaned.csv
|       ├── policies_cleaned.csv
|       ├── vehicles_cleaned.csv
|       ├── claims_cleaned.csv
|       └── payments_cleaned.csv
|
├── notebooks/
|   └── insurance_analysis.ipynb
|
├── src/
|   ├── __init__.py
|   ├── data_loader.py
|   ├── validation.py
|   ├── data_cleaner.py
|   ├── insurance_analysis.py
|   ├── visualization.py
|   ├── insights.py
|   └── logger.py
|
├── app/
|   └── streamlit_app.py
|
├── reports/
|   └── business_report.md
|
├── tests/
|   ├── test_pipeline.py
|   ├── test_analysis.py
|   └── test_validation.py
|
├── logs/
|   └── pipeline.log
|
├── run_pipeline.py
├── requirements.txt
├── Dockerfile
├── .dockerignore
├── .gitignore
└── README.md
```

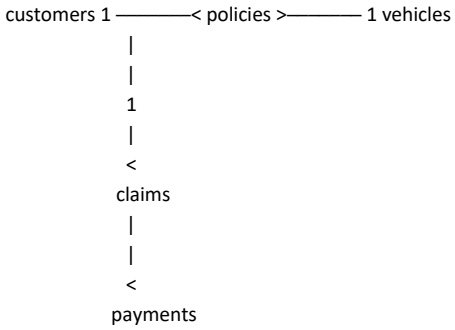
Standardization rule

Students must not rename the standard files, tables, or required functions. They may add comments, markdown explanations, helper functions where appropriate, and their own implementation logic. The interface must remain standardized so the trainer can inspect every project consistently.

6. DATA MODEL — MULTIPLE RELATED TABLES

Table	Primary Key	Important relationships	Purpose
customers	customer_id	customer_id → policies	Customer profile and demographics
vehicles	vehicle_id	vehicle_id → policies	Vehicle attributes
policies	policy_id	customer_id, vehicle_id	Policy, premium and coverage information
claims	claim_id	policy_id, customer_id	Claim event, amount and status
payments	payment_id	claim_id, policy_id	Claim/payment settlement information

Relationship model



Required standard columns

Table	Required columns
customers	customer_id, customer_name, gender, date_of_birth, age, city, state, occupation
vehicles	vehicle_id, customer_id, vehicle_type, vehicle_make, vehicle_model, vehicle_year, fuel_type, vehicle_value
policies	policy_id, customer_id, vehicle_id, policy_start_date, policy_end_date, policy_type, premium_amount, coverage_amount, policy_status, payment_frequency
claims	claim_id, policy_id, customer_id, claim_date, accident_date, claim_type, accident_location, claim_amount, claim_status, approval_date, settlement_date, damage_severity
payments	payment_id, claim_id, policy_id, payment_date, payment_amount, payment_status, payment_method

7. DATA DICTIONARY — CORE FIELDS

Field	Meaning	Type
customer_id	Unique customer identifier	String
vehicle_id	Unique vehicle identifier	String
policy_id	Unique policy identifier	String
claim_id	Unique claim identifier	String
premium_amount	Premium charged for policy	Numeric
coverage_amount	Insured coverage amount	Numeric
claim_amount	Amount requested for claim	Numeric
payment_amount	Amount paid toward claim	Numeric
vehicle_value	Estimated vehicle value	Numeric
policy_status	Active/Expired/Cancelled/Renewed	Categorical
claim_status	Pending/Approved/Rejected/Settled	Categorical
damage_severity	Low/Medium/High	Categorical

8. DERIVED COLUMNS

Derived column	Definition	Business use
policy_duration_days	policy_end_date - policy_start_date	Policy duration analysis
customer_age	Calculated from date_of_birth	Customer segmentation
vehicle_age	Analysis date/year - vehicle_year	Vehicle risk pattern
claim_processing_days	approval_date - claim_date	Approval efficiency
claim_settlement_days	settlement_date - claim_date	Settlement performance
claim_to_premium_ratio	claim amount / premium amount	Cost/risk indicator
claim_to_coverage_ratio	claim amount / coverage amount	Claim severity indicator
paid_claim_ratio	payment amount / claim amount	Settlement/payment analysis
policy_active_flag	1 if policy is active, else 0	Active portfolio analysis

9. BRD — BUSINESS REQUIREMENTS

BR-01 Portfolio

Management must be able to understand the size and status of the policy portfolio.

BR-02 Claims

Management must be able to measure claim volume, claim amount, status, severity, and settlement performance.

BR-03 Customers

Management must be able to compare customer groups and identify meaningful claim and premium patterns.

BR-04 Vehicles

Management must be able to compare claim behavior across vehicle types, makes, fuel types, and vehicle age.

BR-05 Cost and risk

Management must be able to compare premium against claim cost and identify groups requiring investigation.

BR-06 Decision support

The dashboard must convert analytical results into understandable patterns, insights, and recommendations.

10. FRD — REQUIRED MODULES

Requirement	Required function	File
Load a table	load_table(file_path)	src/data_loader.py
Load all tables	load_all_data(data_dir)	src/data_loader.py
Validate tables	validate_all_tables(data)	src/validation.py
Clean data	clean_data(data)	src/data_cleaner.py
Create derived columns	create_derived_columns(data)	src/data_cleaner.py
Save cleaned tables	save_cleaned_data(data, output_dir)	src/data_cleaner.py
Calculate KPIs	Required KPI functions	src/insurance_analysis.py
Create charts	Required plot functions	src/visualization.py
Generate insights	Required insight functions	src/insights.py
Logging	setup_logger()	src/logger.py
Pipeline	run_pipeline()	run_pipeline.py

11. DATA LOADING FUNCTIONS

src/data_loader.py

```
check_file_exists(file_path)
load_table(file_path)
load_all_data(data_dir)
```

load_all_data() should return the standardized set of related DataFrames. The implementation should make the table relationships clear and should not silently ignore missing required files.

12. VALIDATION FUNCTIONS

src/validation.py

```
validate_customers(df)
validate_vehicles(df)
validate_policies(df)
validate_claims(df)
validate_payments(df)
validate_relationships(data)
validate_all_tables(data)
```

- Check required columns.
- Check primary-key uniqueness where required.
- Check missing identifiers.
- Check valid numeric ranges.
- Check valid dates.
- Check policy/claim/payment relationships.
- Check categorical values against the project's defined business rules.

13. DATA CLEANING FUNCTIONS

src/data_cleaner.py

```
handle_missing_values(data)
remove_duplicates(data)
clean_text_columns(data)
clean_date_columns(data)
validate_numeric_columns(data)
validate_business_rules(data)
create_derived_columns(data)
clean_data(data)
save_cleaned_data(data, output_dir)
```

Every function must have a docstring. One function should have one clear responsibility. Students must not place the complete project logic into one large function.

14. CLEANING PIPELINE

```
load_all_data()
    ↓
validate_all_tables()
    ↓
clean_data()
    ↓
handle_missing_values()
    ↓
remove_duplicates()
    ↓
clean_text_columns()
    ↓
```

```
clean_date_columns()
    ↓
validate_numeric_columns()
    ↓
validate_business_rules()
    ↓
create_derived_columns()
    ↓
save_cleaned_data()
```

Pipeline entry point: run_pipeline.py. Required function: run_pipeline(). Expected execution: python run_pipeline.py.

15. STANDARD BUSINESS ANALYSIS FUNCTIONS

src/insurance_analysis.py

```
# Portfolio
calculate_total_customers()
calculate_total_policies()
calculate_active_policies()
calculate_expired_policies()
calculate_cancelled_policies()
calculate_renewed_policies()

# Premium
calculate_total_premium()
calculate_average_premium()
calculate_average_coverage()
calculate_premium_by_policy_type()

# Claims
calculate_total_claims()
calculate_approved_claims()
calculate_rejected_claims()
calculate_pending_claims()
calculate_total_claim_amount()
calculate_average_claim_amount()
calculate_claim_approval_rate()
calculate_claim_rejection_rate()

# Settlement
calculate_average_claim_processing_days()
calculate_average_claim_settlement_days()
calculate_total_payment_amount()
calculate_average_payment_amount()

# Risk / ratios
calculate_claim_to_premium_ratio()
calculate_claim_to_coverage_ratio()
calculate_paid_claim_ratio()

# Customer / vehicle
analyze_customer_claims()
analyze_customer_premium()
analyze_vehicle_claims()
analyze_vehicle_risk_patterns()

# Business analysis
analyze_claim_type()
analyze_claim_status()
analyze_policy_type()
```

analyze_region()
analyze_vehicle_type()
analyze_damage_severity()

16. KPI DEFINITIONS

KPI	Definition
Total Policies	Number of unique policies
Active Policies	Policies whose status is Active
Total Premium	Sum of premium_amount
Average Premium	Average premium per policy
Total Claims	Number of unique claims
Total Claim Amount	Sum of claim_amount
Average Claim Amount	Average claim amount
Claim Approval Rate	Approved claims / total claims × 100
Claim Rejection Rate	Rejected claims / total claims × 100
Average Settlement Days	Average settlement_date - claim_date
Claim-to-Premium Ratio	Claim amount / premium amount
Paid Claim Ratio	Payment amount / claim amount

17. BUSINESS QUESTIONS

Area	Required questions
Policy	How many policies exist? Which policy types/statuses dominate?
Premium	Which policy type generates the highest premium? What is average premium?
Claims	How many claims are filed, approved, rejected and pending?
Claim severity	Which claim types/regions have the highest average claim amount?
Settlement	Which groups have the longest settlement time?
Customer	Which customer segments show higher claim activity or premium?
Vehicle	Which vehicle types/makes/age groups show higher claim patterns?
Geography	Which states/cities have higher claim frequency or severity?
Risk	Which policy/vehicle groups have high claim-to-premium ratios?
Operations	Where should management investigate further?

18. EDA REQUIREMENTS

Univariate

- Premium distribution
- Claim amount distribution
- Vehicle age
- Customer age
- Policy status
- Claim status
- Claim type
- Vehicle type

Bivariate

- Premium vs policy type
- Claim amount vs vehicle type
- Vehicle age vs claim amount
- Customer age vs claim frequency
- Region vs claim amount
- Claim status vs settlement time

Multivariate

- Policy type + vehicle type + claim amount
- Region + claim type + claim severity
- Vehicle age + vehicle type + claim amount
- Customer age + policy type + premium + claims

Charts must have a business purpose. Do not create random charts simply to increase chart count.

19. REQUIRED VISUALIZATIONS

```
plot_policy_status()
plot_policy_type_distribution()
plot_premium_by_policy_type()
plot_monthly_claims()
plot_claim_status()
plot_claim_type_distribution()
plot_claim_amount_distribution()
plot_claim_amount_by_vehicle_type()
plot_claims_by_region()
plot_claim_severity_by_region()
plot_average_settlement_time()
plot_claim_to_premium_by_policy_type()
plot_vehicle_age_vs_claim_amount()
plot_customer_age_vs_claim_amount()
plot_vehicle_type_claim_performance()
plot_damage_severity_distribution()
plot_payment_status()
plot_monthly_premium_trend()
```

20. VISUALIZATION FUNCTION RULE

Every visualization function must receive the required DataFrame(s), perform only the required aggregation, create the chart, add a meaningful title, label axes, and display or return the chart.

21. INSIGHT AND PATTERN ENGINE

This is a key difference in the new project. The Streamlit frontend must not be only a collection of charts. It must surface important patterns from the analysis.

```
src/insights.py

generate_portfolio_insights(data)
generate_claim_insights(data)
generate_premium_insights(data)
generate_customer_insights(data)
generate_vehicle_insights(data)
```

```
generate_risk_patterns(data)
generate_business_recommendations(data)
```

Examples of acceptable pattern statements: 'A small set of vehicle groups contributes a disproportionately high share of claim value'; 'one region has a higher claim frequency than the portfolio average'; or 'a policy segment has a high claim-to-premium ratio'. The actual values and conclusions must come from the assigned data.

22. REQUIRED INSIGHT FORMAT

```
PATTERN
-------
What unusual, important, increasing/decreasing, high/low, or concentrated behavior was observed?

EVIDENCE
--------
What metric, comparison, or chart supports it?

BUSINESS MEANING
----------------
Why does the pattern matter to an insurer?

RECOMMENDATION
--------------
What should the business investigate or improve?
```

23. STREAMLIT FRONTEND

```
app/streamlit_app.py

load_dashboard_data()
show_sidebar_filters()
show_overview()
show_policy_analysis()
show_claim_analysis()
show_customer_analysis()
show_vehicle_analysis()
show_premium_analysis()
show_insights()
main()
```

Dashboard pages

Page	Required content
Executive Overview	Total policies, active policies, total premium, total claims, total claim amount, approval rate, average settlement days
Policy Analysis	Policy status, policy type, premium patterns, monthly trends
Claims Analysis	Claim volume, claim status, claim type, severity, claim amount
Customer Analysis	Customer segments, age patterns, premium and claim behavior
Vehicle Analysis	Vehicle type, make, age, fuel type and claim patterns
Premium & Risk	Premium, claim-to-premium ratio, coverage, risk patterns
Insights & Recommendations	Top patterns, evidence, business meaning and recommended actions

Sidebar filters

- Policy Type
- Policy Status
- Claim Status

- Claim Type
- Vehicle Type
- Vehicle Make
- State/City
- Damage Severity
- Date range where applicable

Filters must update the displayed KPIs, charts, patterns, and insights.

24. FRONTEND INSIGHT DESIGN

The final frontend should contain visible insight/pattern cards alongside analytical visuals.

Example frontend:

KPI CARD
Total Claims: 1,245

PATTERN
High claim concentration in Vehicle Type X

EVIDENCE
Vehicle Type X contributes 31% of total claim amount.

BUSINESS MEANING
This segment is contributing disproportionately to claim cost.

RECOMMENDATION
Review claim characteristics, vehicle age, damage severity,
and underwriting/operational factors for this segment.

Students must not hard-code fake findings. The displayed insight must be calculated from the active dataset/filter selection.

25. NOTEBOOK STRUCTURE

1. Business Understanding
2. Import Libraries
3. Load Related Tables
4. Understand Table Relationships
5. Basic Data Understanding
6. Data Quality Analysis
7. Data Cleaning
8. Derived Columns
9. EDA — Univariate
10. EDA — Bivariate
11. EDA — Multivariate
12. Statistical Analysis
13. KPI Analysis
14. Business Questions
15. Visualizations
16. Patterns
17. Business Insights
18. Recommendations
19. Final Conclusion

26. STATISTICAL ANALYSIS

- Mean, median and mode for premium and claim amount.
- Variance and standard deviation for premium, claim amount and settlement time.
- Correlation between relevant numeric variables.
- Distribution analysis for claim amount and premium.
- Comparison of groups using appropriate descriptive statistics.

The project is beginner → intermediate. Statistical analysis should support business understanding rather than become a separate mathematical exercise.

27. TESTING — PYTEST

tests/test_pipeline.py
test_load_all_data()
test_remove_duplicates()
test_create_derived_columns()
test_clean_data()
test_save_cleaned_data()

tests/test_analysis.py
test_calculate_total_policies()
test_calculate_active_policies()
test_calculate_total_claims()
test_calculate_total_premium()
test_calculate_claim_approval_rate()
test_calculate_average_claim_amount()

tests/test_validation.py
test_validate_customers()
test_validate_policies()
test_validate_claims()
test_validate_relationships()

Tests should verify expected behavior using small controlled DataFrames or test fixtures. Students must be able to explain what each test proves.

28. LOGGING

src/logger.py

setup_logger()
log_pipeline_step(message)
log_error(message)

- Record pipeline start and completion.
- Record table loading.
- Record validation stages.
- Record cleaning stages.
- Record output generation.
- Record errors with useful messages.

Logs should be useful for troubleshooting. Do not log secrets, credentials, access keys, or sensitive user information.

29. GIT / GITHUB

- Create a Git repository for the project.
- Commit meaningful milestones rather than one final upload only.
- Keep data and secrets according to the project rules and .gitignore.
- README must explain setup, pipeline execution, dashboard execution, testing and deployment.
- Student must be able to explain their own commit history and project structure.

30. DOCKER

Dockerfile requirements:

1. Use a suitable Python base image.
2. Set a working directory.
3. Copy requirements.txt.
4. Install dependencies.
5. Copy application source.
6. Expose Streamlit port 8501.
7. Start the Streamlit application.

Build:
docker build -t motor-insurance-analytics .

Run locally:
docker run -p 8501:8501 motor-insurance-analytics

31. AWS ECR

- Create an Amazon ECR repository for the application.
- Authenticate Docker to ECR.
- Tag the local image with the ECR repository URI.
- Push the image to ECR.
- Verify the image is available in ECR.

Students must understand what ECR stores and why a container registry is used. They must not simply execute commands without being able to explain them.

32. AWS IAM

- Use an IAM role/user configuration appropriate to the deployment setup.
- Follow least-privilege principles.
- Do not put AWS access keys directly in source code or Docker images.
- The student must explain which AWS permissions are required and why.

33. AWS EC2 DEPLOYMENT

High-level deployment flow:

Docker image
↓
Amazon ECR
↓
EC2 instance
↓
Pull image from ECR
↓

Run Docker container
↓
Streamlit on port 8501
↓
Security Group allows required application traffic
↓
Public application endpoint

- Launch and configure an EC2 instance suitable for the project.
- Install/configure Docker.
- Provide the EC2 instance access to pull the ECR image using the approved IAM approach.
- Pull the required image.
- Run the Streamlit container.
- Configure the network/security group for the application port.
- Verify the application from an external browser.
- Record the final live application URL.

34. LIVE URL SUBMISSION

After successful AWS deployment, every student must obtain the working Streamlit application URL and submit that URL in the Google Form provided by the admin. The Google Form is already created by the admin; students must use the form provided to them.

Submission sequence:

Project completed
↓
GitHub repository ready
↓
Docker image built
↓
Image pushed to ECR
↓
Application running on EC2
↓
Open live Streamlit URL in browser
↓
Verify dashboard + filters + insights
↓
Copy live URL
↓
Submit URL in Admin Google Form

35. README REQUIREMENTS

- Project title
- Business scenario
- Business problem
- Objectives
- Technology stack
- Data model and table relationships
- Project structure
- Installation
- Data cleaning pipeline
- How to run tests

- How to run Streamlit locally
- Docker instructions
- AWS deployment overview
- Key KPIs
- Business questions
- Important insights
- Recommendations
- Live application URL

36. FINAL BUSINESS REPORT

- Executive Summary
- Business Problem
- Data and Table Summary
- Data Quality Findings
- Cleaning Performed
- KPI Summary
- EDA Findings
- Important Patterns
- Business Insights
- Recommendations
- Dashboard Summary
- Deployment Summary
- Conclusion

37. ACCEPTANCE CRITERIA

- Standard project structure is followed.
- All required tables are present and related correctly.
- Raw data is preserved.
- Validation works.
- Cleaning pipeline runs successfully.
- Cleaned data is generated.
- Derived columns are created correctly.
- Notebook runs from beginning to end.
- Required KPIs are calculated.
- Required business questions are answered.
- Required visualizations are created.
- Patterns are identified from actual data.
- Insights contain evidence and business meaning.
- Recommendations are evidence-based.
- Streamlit dashboard works.
- Filters update results.
- Insights update with the active dataset/filter context.
- Pytest tests pass.
- Logging is implemented.
- Git repository is usable.
- Docker image builds and runs.
- Image is pushed to ECR.
- Application is deployed on EC2.

- Live URL works externally.
- Live URL is submitted through the admin Google Form.
- README and business report are complete.

38. STUDENT CHECKLIST

[ ] Folder structure correct
[ ] File names correct
[ ] Table names correct
[ ] Required columns correct
[ ] Relationships understood
[ ] Validation implemented
[ ] Cleaning implemented
[ ] Derived columns implemented
[ ] Pipeline runs
[ ] EDA completed
[ ] Statistics completed
[ ] KPIs completed
[ ] Business questions answered
[ ] Required charts completed
[ ] Patterns identified
[ ] Insights generated
[ ] Recommendations written
[ ] Streamlit works
[ ] Filters work
[ ] Frontend insights work
[ ] Pytest passes
[ ] Logging works
[ ] GitHub repository complete
[ ] Docker build works
[ ] ECR image pushed
[ ] EC2 deployment works
[ ] Live URL verified
[ ] Live URL submitted in Admin Google Form
[ ] README complete
[ ] Business report complete

39. TRAINER / INTERVIEW EVALUATION CHECKPOINTS

Because the project interface is standardized, the trainer can evaluate understanding by opening any student's project and tracing the same path.

Checkpoint	Question to ask
Data model	Why are claims connected to policy_id?
Validation	What relationship validation did you perform?
Cleaning	Why did you handle this missing value this way?
Derived field	Why did you create claim_to_premium_ratio?
Function	What is the responsibility of this function?
KPI	How is claim approval rate calculated?
EDA	Why did you choose this chart?
Pattern	How did you identify this pattern?
Insight	What evidence supports your statement?
Recommendation	Why does this recommendation follow from the evidence?
Streamlit	How do filters affect the calculations?
Testing	What does this pytest test prove?
Logging	What happens when the pipeline fails?
Docker	Why do we need Docker here?
ECR	Why is the image stored in ECR?

IAM	Why does EC2 need the required AWS permission?
EC2	How does the browser reach your Streamlit app?
Deployment	Show the live URL and explain the deployment flow.

40. CORE STANDARDIZATION RULE

SAME BUSINESS DOMAIN
+
SAME TABLE DESIGN
+
SAME FOLDER STRUCTURE
+
SAME FILE NAMES
+
SAME FUNCTION NAMES
+
SAME KPI DEFINITIONS
+
SAME BUSINESS QUESTIONS
+
SAME MINIMUM VISUALIZATIONS
+
SAME TESTING INTERFACE
+
SAME DEPLOYMENT TARGET

BUT

STUDENT'S OWN IMPLEMENTATION
+
STUDENT'S OWN ANALYSIS
+
STUDENT'S OWN PATTERNS
+
STUDENT'S OWN INSIGHTS
+
STUDENT'S OWN RECOMMENDATIONS

41. FINAL PROJECT STORY

At the end of the internship, a student should be able to explain: "I worked on a motor insurance analytics project. I started from business requirements and converted them into a standardized analytics solution. I worked with related customer, vehicle, policy, claim, and payment tables, validated their relationships, built a reusable cleaning pipeline, performed EDA and statistical analysis, calculated insurance KPIs, answered business questions, identified patterns, converted those patterns into business insights and recommendations, and built a Streamlit dashboard. I then tested the application, added logging, containerized it with Docker, pushed the image to Amazon ECR, deployed it on AWS EC2, verified the live application, and submitted the live URL through the admin Google Form."